

Strong Contracts as the Control Surface for AI-Written Python

The central argument for Axiomander is not that contracts should describe small facts like “this integer is positive”. The argument is that contracts should describe the *important activities of the program*: what resources are owned, what state may change, what invariants are preserved, what external effects are permitted, and what domain-level outcome has been achieved.

If AI writes most of the implementation, the contract becomes the human-readable control surface. The AI may choose algorithms, decomposition, helper functions, data structures, and refactorings, but it must satisfy a top-level specification that is strong enough to express the real purpose of the software.

Example: exactly-once order fulfilment

```
def fulfil_order(order_id: OrderId, worker_id: WorkerId) -> FulfilmentResult:
    """
    axiomander:
        requires OrderQueue.contains(order_id)
        requires Order(order_id).status == "ready"
        requires Payment(order_id).state == "authorized"
        requires Inventory.can_reserve(Order(order_id).items)

        owns queue_item: OrderQueue.item(order_id)
        owns order_row: Orders.row(order_id)
        owns payment_auth: Payment.authorization(order_id)
        owns stock: Inventory.reservation_rights(Order(order_id).items)

        frame:
            may_modify Orders.row(order_id)
            may_modify OrderQueue.item(order_id)
            may_modify Inventory.reservations(for_order=order_id)
            may_modify Payment.capture(order_id)
            may_emit EventBus.topic("orders.fulfilled")
            must_not_modify Orders.rows(except=order_id)
            must_not_modify Inventory.stock_totals(except=Order(order_id).items)
            must_not_emit EventBus.topic(except="orders.fulfilled")

        ensures result.status in {"fulfilled", "failed_recoverably"}

        ensures result.status == "fulfilled" ->
            Orders.row(order_id).status == "fulfilled"
            and Payment(order_id).state == "captured"
            and Inventory.reserved_for(order_id, Order(order_id).items)
```

```

    and OrderQueue.item(order_id).state == "completed"
    and exists e in EventBus.emitted:
        e.topic == "orders.fulfilled"
        and e.payload.order_id == order_id
        and e.payload.worker_id == worker_id

ensures result.status == "failed_recoverably" ->
    Orders.row(order_id).status in {"ready", "fulfilment_pending"}
    and Payment(order_id).state in {"authorized", "capture_pending"}
    and OrderQueue.item(order_id).state in {"ready", "retry"}
    and no_lost_inventory(Order(order_id))

ensures exactly_once_domain_effect(order_id):
    forall histories h:
        count(successful_fulfilments(h, order_id)) <= 1

preserves GlobalInvariant.accounting_consistency
preserves GlobalInvariant.inventory_nonnegative
preserves GlobalInvariant.queue_order_correspondence
"""

```

This contract is complex because the operation is complex. The real requirement is not “return a FulfilmentResult”. The real requirement is that payment, inventory, order state, queue state, and emitted events move together without duplication or loss.

The top-level contract motivates the subcontracts:

```

def capture_payment(order_id: OrderId) -> CaptureResult:
    """
    axiomander:
        requires Payment(order_id).state == "authorized"
        owns Payment.authorization(order_id)

    frame:
        may_modify Payment.capture(order_id)
        must_not_modify Orders.any()
        must_not_modify Inventory.any()

    ensures result.ok ->
        Payment(order_id).state == "captured"

    ensures not result.ok ->
        Payment(order_id).state in {"authorized", "capture_pending"}

    ensures idempotent_effect:
        forall n >= 1:

```

```

        repeated_calls(n, capture_payment(order_id))
        causes at_most_one_external_capture(order_id)
    """

```

```

def reserve_inventory(order_id: OrderId, items: list[LineItem]) ->
ReservationResult:
    """
    axiomander:
        requires forall item in items:
            Inventory.available(item.sku) >= item.quantity

        owns Inventory.reservation_rights(items)

    frame:
        may_modify Inventory.reservations(for_order=order_id)
        may_modify Inventory.available_counts(skus(items))
        must_not_modify Payment.any()
        must_not_modify Orders.rows(except=order_id)

    ensures result.ok ->
        forall item in items:
            Inventory.reserved(order_id, item.sku) == item.quantity

    ensures result.ok ->
        forall sku:
            Inventory.available(sku) >= 0

    preserves InventoryInvariant.no_negative_stock
    preserves InventoryInvariant.reserved_plus_available_equals_total
    """

```

Example: database migration with safety guarantees

```

def migrate_customer_addresses(batch_id: BatchId) -> MigrationReport:
    """
    axiomander:
        requires Migration(batch_id).state == "planned"
        requires SourceTable("customers").schema_version == 3
        requires TargetTable("customer_addresses").schema_version == 1

        owns migration_lock: MigrationLock(batch_id)
        owns source_snapshot: Snapshot(SourceTable("customers"))
        owns target_write_region: TableRegion("customer_addresses",
batch=batch_id)
    """

```

```

    frame:
      may_modify TargetTable("customer_addresses").rows(where batch_id ==
batch_id)
      may_modify Migration(batch_id).state
      may_emit AuditLog.entries(kind="migration")
      must_not_modify SourceTable("customers").any()
      must_not_modify TargetTable("customer_addresses").rows(where batch_id !=
batch_id)

    ensures result.completed ->
      Migration(batch_id).state == "completed"
      and forall c in SourceTable("customers").snapshot(source_snapshot):
        exists a in TargetTable("customer_addresses"):
          a.customer_id == c.id
          and a.address == normalize_address(c.address)
          and a.batch_id == batch_id

    ensures result.completed ->
      no_duplicate_addresses(batch_id)

    ensures not result.completed ->
      Migration(batch_id).state in {"planned", "partial", "failed"}
      and recoverable_migration_state(batch_id)

    preserves DatabaseInvariant.primary_keys_unique
    preserves DatabaseInvariant.foreign_keys_valid
    preserves AuditInvariant.every_write_has_audit_entry
    """

```

Here the contract justifies quantification. A migration is only correct if *all* relevant source rows have corresponding target rows. It justifies frames because the most dangerous bug in a migration is often unintended modification of unrelated data.

Example: concurrent lease-based worker

```

def process_next_job(worker_id: WorkerId) -> JobOutcome:
    """
    axiomander:
      requires Worker(worker_id).enabled
      requires Queue("jobs").available_or_leased()

    acquire lease:
      exists job_id:
        Queue("jobs").lease(job_id, worker_id, ttl=LEASE_TTL)

    owns leased_job: Queue("jobs").leased_by(worker_id)
    """

```

```

owns job_effect_region: JobEffects.for_leased_job(worker_id)

frame:
  may_modify Queue("jobs").leased_by(worker_id)
  may_modify JobState(of_leased_job(worker_id))
  may_emit EventBus.topic("jobs.completed")
  may_emit EventBus.topic("jobs.failed")
  must_not_modify Queue("jobs").items(not_leased_by=worker_id)
  must_not_modify JobState(not_leased_by=worker_id)

ensures result.kind == "no_job" ->
  not exists job_id:
    Queue("jobs").was_available_to(worker_id, job_id)

ensures result.kind == "completed" ->
  JobState(result.job_id).status == "completed"
  and Queue("jobs").item(result.job_id).state == "acknowledged"
  and exactly_once_job_effect(result.job_id)

ensures result.kind == "retry" ->
  Queue("jobs").item(result.job_id).state in {"ready", "leased"}
  and JobState(result.job_id).status in {"pending", "retryable_failure"}

concurrent invariant:
  forall j:
    count_active_leases(j) <= 1

preserves QueueInvariant.no_duplicate_active_leases
preserves QueueInvariant.completed_jobs_not_reprocessed
"""

```

This is the kind of contract that makes AI-written concurrency plausible. The AI does not merely write code that “probably works”; it writes code under a specification saying which lease it owns, what it may touch, what it must not touch, and what global concurrency invariant must survive.

Why the complexity is justified

The contract is complex because the software’s correctness condition is complex. Real software rarely fails because a single local variable has the wrong type. It fails because state changes are not coordinated: payment succeeds but the order is not marked fulfilled; inventory is decremented twice; a queue item is retried after completion; a migration rewrites the wrong rows.

Strong contracts let us express the real boundary of correctness:

1. **Preconditions** say when an operation is allowed.
2. **Postconditions** say what domain fact has been achieved.

3. **Frames** say what the function may and may not change.
4. **Ownership/resource clauses** say which state this call controls.
5. **Quantifiers** express whole-database or whole-collection correctness.
6. **Invariants** connect local operations to global system safety.
7. **Effect clauses** constrain queues, databases, logs, APIs, and event buses.

The top-level contract should be written first. It explains why the helper contracts exist. Each sub-function then proves a smaller part of the top-level story.

This is the engineering model for AI-written software:

```
human intent
  -> strong top-level contract
    -> AI-generated decomposition
      -> AI-generated helper contracts
        -> proof obligations
          -> SMT / theorem prover / symbolic execution / reviewed axioms
```

The AI may write most of the code and many of the contracts, but it is not unconstrained. It is working inside a specification graph rooted in a human-approved top-level property. The implementation can change freely only where the contract permits it. Refactoring becomes safe because the proof obligations are regenerated. Helper functions become trustworthy because their contracts are justified by the parent contract.

The goal is not to remove human judgement. The goal is to move human judgement upward: from reviewing every line of Python to reviewing the contracts that describe what the software is allowed to do.